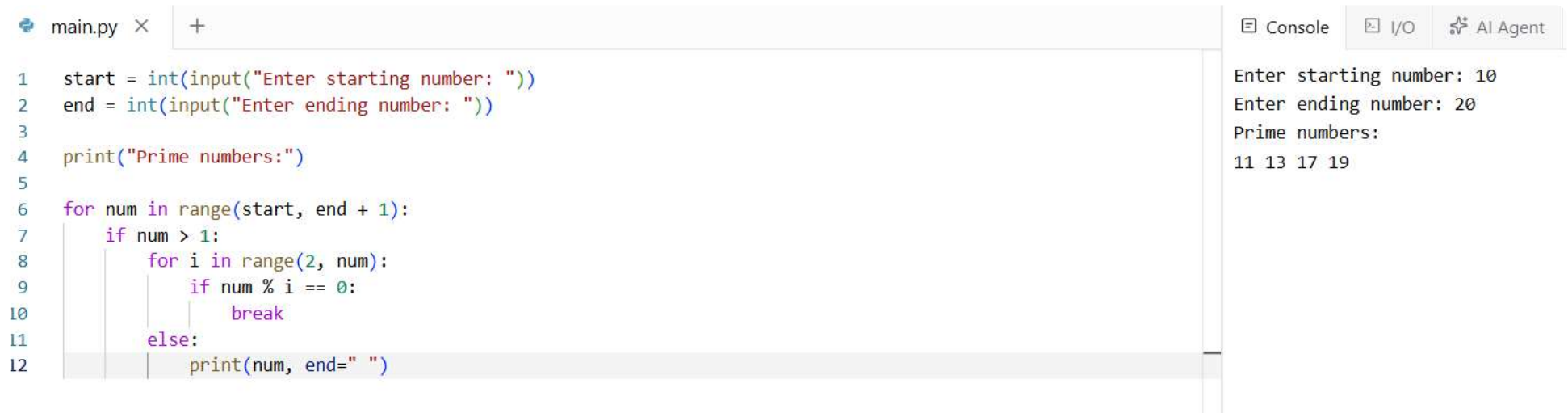


1. Print all prime numbers between input range (Ex – input 20 50, prints all prime numbers between 20 and 50).

```
start = int(input("Enter starting number: "))
end = int(input("Enter ending number: "))
print("Prime numbers:")

for num in range(start, end + 1):
    if num > 1:
        for i in range(2, num):
            if num % i == 0:
                break
        else:
            print(num, end=" ")
```

output:



```
main.py x +
```

```
1 start = int(input("Enter starting number: "))
2 end = int(input("Enter ending number: "))
3
4 print("Prime numbers:")
5
6 for num in range(start, end + 1):
7     if num > 1:
8         for i in range(2, num):
9             if num % i == 0:
10                break
11        else:
12            print(num, end=" ")
```

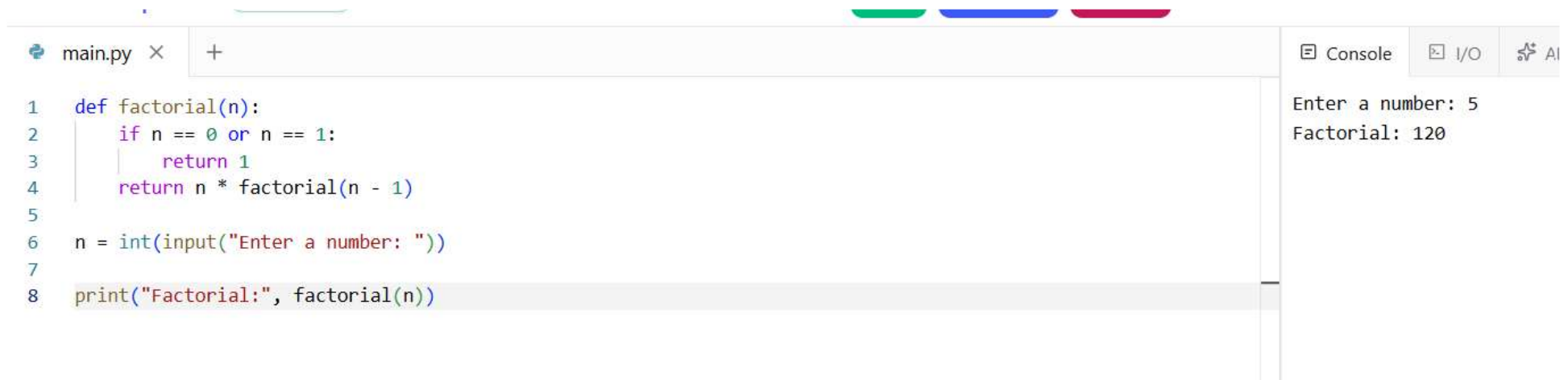
Console I/O AI Agent

Enter starting number: 10
Enter ending number: 20
Prime numbers:
11 13 17 19

2.Factorial using recursion

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)  
  
n = int(input("Enter a number: "))  
  
print("Factorial:", factorial(n))
```

output:



The screenshot shows a code editor with a file named 'main.py'. The code defines a recursive function 'factorial(n)' that returns 1 for n=0 or n=1, and n * factorial(n-1) otherwise. It then prompts the user to 'Enter a number: ' and prints the result as 'Factorial: 120'. The console output on the right shows the user input '5' and the program output 'Factorial: 120'.

```
1 def factorial(n):  
2     if n == 0 or n == 1:  
3         return 1  
4     return n * factorial(n - 1)  
5  
6 n = int(input("Enter a number: "))  
7  
8 print("Factorial:", factorial(n))
```

Console
I/O
AI

Enter a number: 5
Factorial: 120

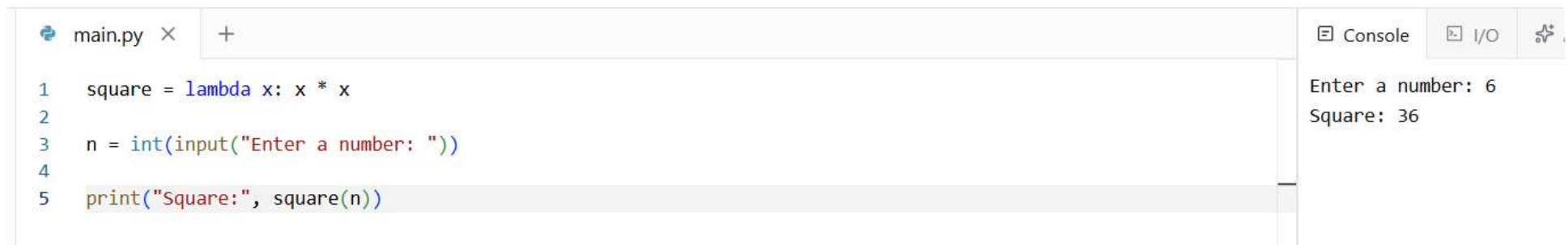
3.Square of numbers using lambda

```
square = lambda x: x * x

n = int(input("Enter a number: "))

print("Square:", square(n))
```

output:

A screenshot of a code editor interface. The editor has a tab labeled 'main.py' with a close button 'x' and a '+' sign for new files. The code is as follows:

```
1 square = lambda x: x * x
2
3 n = int(input("Enter a number: "))
4
5 print("Square:", square(n))
```

The line numbers 1 through 5 are on the left. The output console on the right shows:

```
Enter a number: 6
Square: 36
```

The console has tabs for 'Console', 'I/O', and a star icon. The 'Console' tab is active.

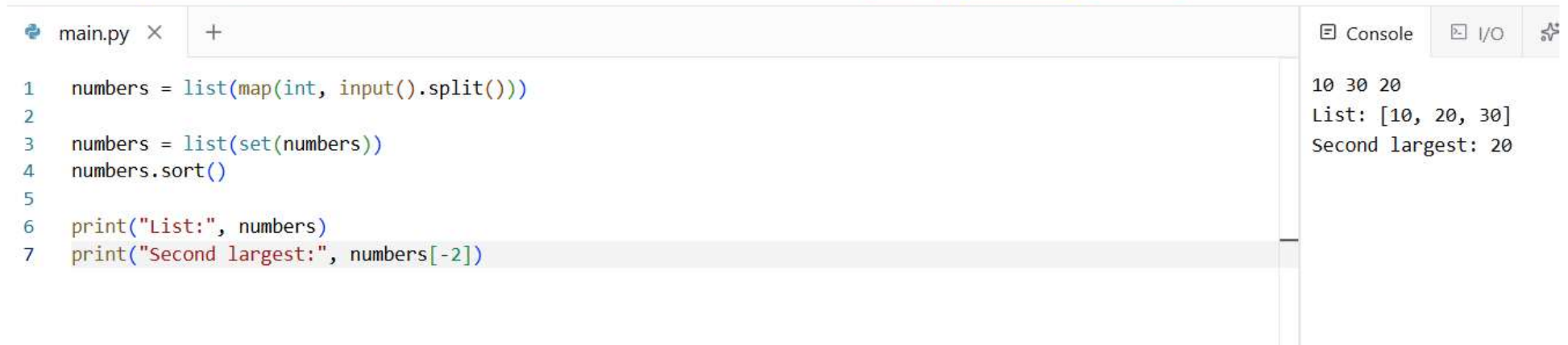
4.Find the second largest element in a list

```
numbers = list(map(int, input().split()))

numbers = list(set(numbers))
numbers.sort()

print("List:", numbers)
print("Second largest:", numbers[-2])
```

output:



The screenshot shows a code editor with a file named 'main.py'. The code in the editor is as follows:

```
1 numbers = list(map(int, input().split()))
2
3 numbers = list(set(numbers))
4 numbers.sort()
5
6 print("List:", numbers)
7 print("Second largest:", numbers[-2])
```

To the right of the code editor is a console window. It displays the output of the program:

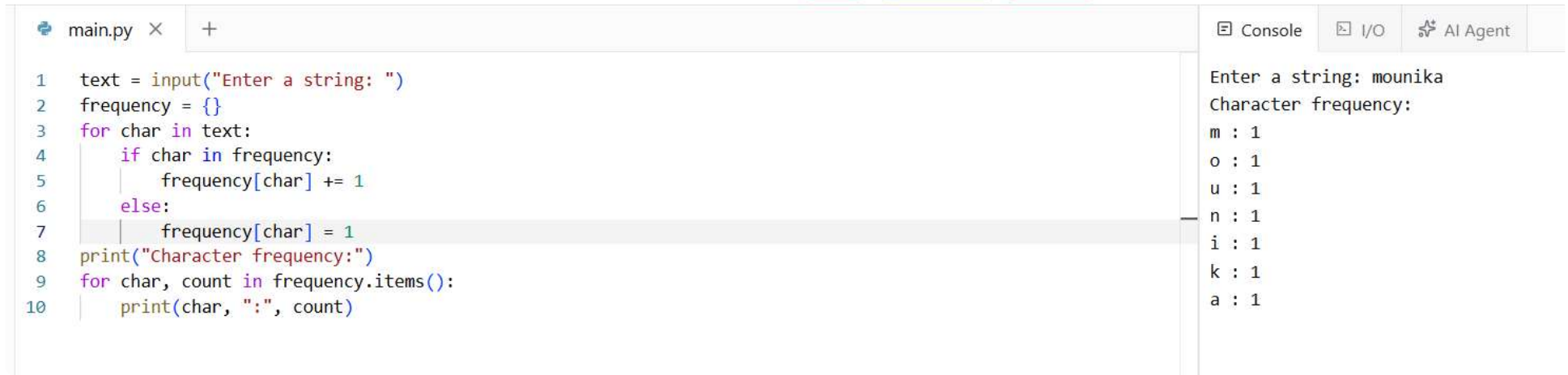
```
10 30 20
List: [10, 20, 30]
Second largest: 20
```

5.Count frequency of characters in a string

```
text = input("Enter a string: ")
frequency = {}
for char in text:
    if char in frequency:
        frequency[char] += 1
    else:
        frequency[char] = 1

print("Character frequency:")
for char, count in frequency.items():
    print(char, ":", count)
```

output:



The screenshot shows a code editor with a file named `main.py`. The code calculates the frequency of characters in a string. The console output shows the user input 'mounika' and the resulting character frequency.

```
1 text = input("Enter a string: ")
2 frequency = {}
3 for char in text:
4     if char in frequency:
5         frequency[char] += 1
6     else:
7         frequency[char] = 1
8 print("Character frequency:")
9 for char, count in frequency.items():
10    print(char, ":", count)
```

Console Output:

```
Enter a string: mounika
Character frequency:
m : 1
o : 1
u : 1
n : 1
i : 1
k : 1
a : 1
```

6. Calculate area of a circle using math library.

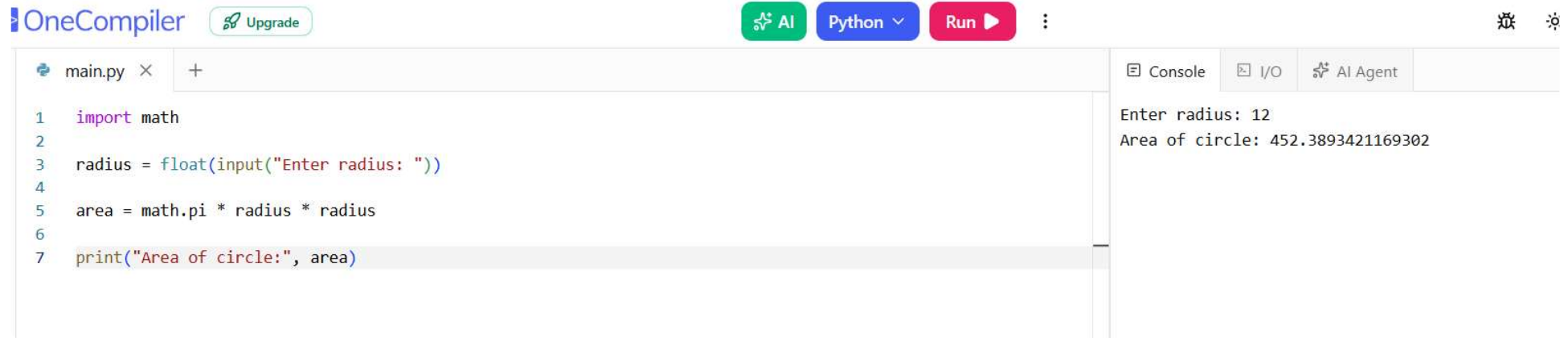
```
import math

radius = float(input("Enter radius: "))

area = math.pi * radius * radius

print("Area of circle:", area)
```

output:



The screenshot shows the OneCompiler Python IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', and a menu icon. Below the header, the editor shows a file named 'main.py' with the following Python code:

```
1 import math
2
3 radius = float(input("Enter radius: "))
4
5 area = math.pi * radius * radius
6
7 print("Area of circle:", area)
```

On the right side, there's a 'Console' tab selected, showing the output of the program:

```
Enter radius: 12
Area of circle: 452.3893421169302
```

7.Reverse a string without using built-in reverse

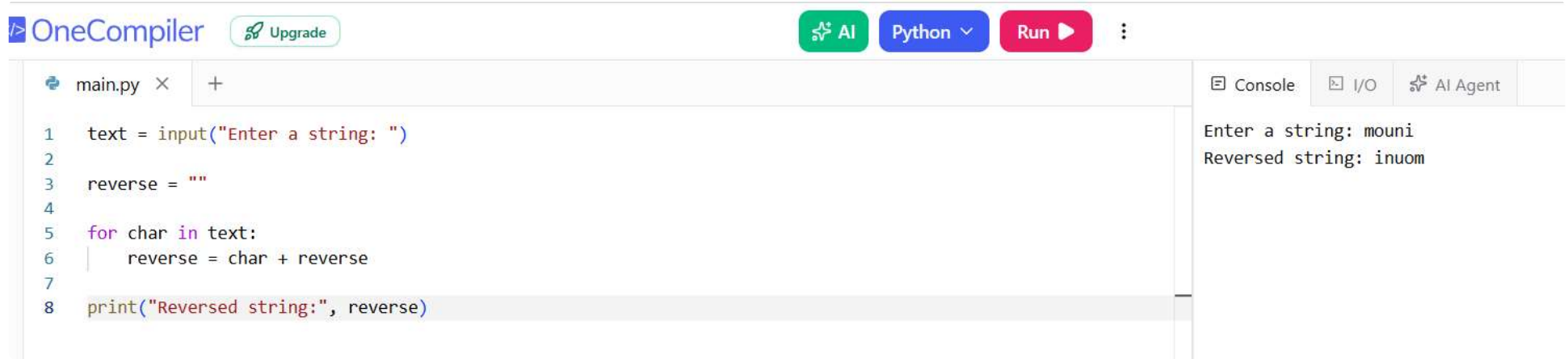
```
text = input("Enter a string: ")
```

```
reverse = ""
```

```
for char in text:
    reverse = char + reverse
```

```
print("Reversed string:", reverse)
```

output:



The screenshot shows the OneCompiler web IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', and a menu icon. Below the header, the editor displays a file named 'main.py' with the following Python code:

```
1 text = input("Enter a string: ")
2
3 reverse = ""
4
5 for char in text:
6     reverse = char + reverse
7
8 print("Reversed string:", reverse)
```

On the right side, there are tabs for 'Console', 'I/O', and 'AI Agent'. The 'Console' tab is active, showing the output of the program:

```
Enter a string: mouni
Reversed string: inuom
```

8.Remove duplicates from a list

```
numbers = [10, 20, 10, 30, 20, 40, 30]
```

```
unique = []
```

```
for num in numbers:
    if num not in unique:
        unique.append(num)
```

```
print("Original list:", numbers)
print("List after removing duplicates:", unique)
```

output:

```
main.py × +  
1 numbers = [10, 20, 10, 30, 20, 40, 30]  
2  
3 unique = []  
4  
5 for num in numbers:  
6     if num not in unique:  
7         unique.append(num)  
8  
9 print("Original list:", numbers)  
10 print("List after removing duplicates:", unique)
```

Console I/O AI Agent

Original list: [10, 20, 10, 30, 20, 40, 30]
List after removing duplicates: [10, 20, 30, 40]

9.Merge two dictionaries

```
dict1 = {"a": 10, "b": 20}  
dict2 = {"c": 30, "d": 40}
```

```
merged = dict1.copy()  
merged.update(dict2)
```

```
print("Dictionary 1:", dict1)  
print("Dictionary 2:", dict2)  
print("Merged dictionary:", merged)
```

output:

```
main.py x +
1 dict1 = {"a": 10, "b": 20}
2 dict2 = {"c": 30, "d": 40}
3
4 merged = dict1.copy()
5 merged.update(dict2)
6
7 print("Dictionary 1:", dict1)
8 print("Dictionary 2:", dict2)
9 print("Merged dictionary:", merged)
```

Console I/O AI Agent

Dictionary 1: {'a': 10, 'b': 20}
Dictionary 2: {'c': 30, 'd': 40}
Merged dictionary: {'a': 10, 'b': 20, 'c': 30, 'd': 40}

10.Fibonacci series using recursion

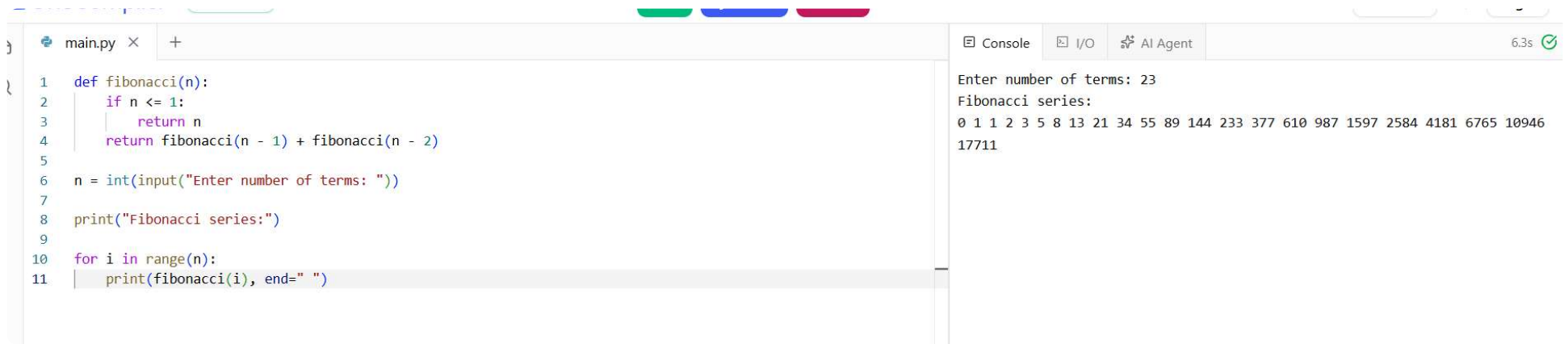
```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number of terms: "))

print("Fibonacci series:")

for i in range(n):
    print(fibonacci(i), end=" ")
```

output:



The screenshot shows a code editor with a file named `main.py`. The code defines a recursive function `fibonacci(n)` and uses it to generate the first 23 terms of the Fibonacci sequence. The console output shows the input '23' and the resulting sequence of numbers.

```
1 def fibonacci(n):
2     if n <= 1:
3         return n
4     return fibonacci(n - 1) + fibonacci(n - 2)
5
6 n = int(input("Enter number of terms: "))
7
8 print("Fibonacci series:")
9
10 for i in range(n):
11     print(fibonacci(i), end=" ")
```

Console Output:

```
Enter number of terms: 23
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946
17711
```